

Technical Evaluation

of
CRT Tabulator Package Replacement

Document Reference	
Version	Initial
Date	3/19/26

Forrest Rawles

Lockheed Martin 2026

Table of Contents:

(U) Definitions.....	5
(U) Labels:.....	5
1. (U) Investigation Results.....	5
1.1 Libraries That Did Not Meet Criteria.....	5
1.1.1 HandsonTable.....	5
1.1.2 Tanstack.....	5
1.1.3 RevoGrid.....	5
2. (U) MUI X Data Grid.....	6
2.1 Introduction:.....	6
2.2 Main Features:.....	6
2.2.1 Cells.....	6
2.2.1.1 renderCell().....	6
2.2.1.2 valueGetter().....	6
2.2.1.3 valueFormatter().....	6
2.2.2 Columns.....	7
2.2.2.1 Column Visibility (C).....	7
2.2.2.2 Column Headers (C).....	7
2.2.2.3 Column Menu (C).....	8
2.2.2.4 Column Groups (C).....	8
2.2.2.5 Column Reordering (Pro).....	9
2.2.2.6 Column Pinning (Pro).....	9
2.2.3 Filtering.....	9
2.2.3.1 Overview.....	9
2.2.3.1.a Excel-Like Filtering (C).....	10
2.2.3.1.b Filter Panel (C)/(I).....	10
2.2.3.2 Header Filters (Pro).....	10
2.2.3.3 Multi-Filters (Pro).....	10
2.2.3.4 Custom Filters.....	11
2.2.3.4.a Custom Input Components (C).....	11
2.2.3.4.b Multiple Values Operator (C).....	11
2.2.3.5 Sorting (C).....	11
2.2.3.6 Multi-Sorting (Pro).....	11
2.2.3.7 Quick Filter (C).....	11
2.2.4 Rows.....	12
2.2.4.1 Density (C).....	12
2.2.4.2 Dynamic Row Heights (C).....	12
2.2.4.3 High Frequency (Pro).....	12
2.2.4.4 Master-Detail Row Panels (Pro).....	13
2.2.4.5 Row Pinning (Pro).....	13

2.2.4.6 Row Reordering (Pro).....	13
2.2.4.7 Row Selections.....	14
2.2.4.7.a Cell Selection (P).....	14
2.2.4.7.b Single Selection (C).....	14
2.2.4.7.b Multi Selection (Pro).....	14
2.2.4.7.c Disable selection on certain rows (C).....	14
2.2.4.7.d Row selection with filtering (C).....	15
2.2.4.8 Row Spanning (C).....	15
2.2.4.9 Column Virtualization (C).....	15
2.3 Advanced Features:.....	16
2.3.1 Aggregation (P).....	16
2.3.2 AI Assistant (P).....	16
2.3.2.1 Overview.....	16
2.3.2.2 Prompt Field.....	17
2.3.2.3 AI Assistant Panel (I).....	17
2.3.3 List View (Pro).....	17
2.3.4 Pivoting (P).....	18
2.3.4.1 Rows.....	18
2.3.4.2 Values.....	18
2.3.4.3 Columns.....	19
2.3.5 Row Grouping (P).....	20
2.3.6 Tree Data (Pro).....	20
2.4 Components:.....	21
2.4.1 Column Panel (I).....	21
2.4.2 Export.....	21
2.4.2.1 CSV (C).....	21
2.4.2.1 Excel (P).....	22
2.4.3 Toolbar.....	23
2.5 Customization:.....	23
2.5.1 Overlays.....	23
2.5.1.1 Skeleton.....	24
2.5.1.2 Linear Progress.....	24
2.5.1.3 Circular Progress.....	24
2.5.1.4 Custom Component.....	24
2.5.1.5 No Columns Overlay.....	24
2.5.1.6 No Rows Overlay.....	24
2.5.1.7 No Results Overlay.....	25
2.6 Concerns:.....	25
2.7 Estimation:.....	25
3. (U) FancyGrid.....	26
3.1 Introduction.....	26

3.2 Features.....	26
3.3 Demo Links.....	27
3.4 Concerns.....	27
3.5 Estimation.....	27
4. (U) Internal CRT Table Component.....	27
4.1 Concerns:.....	27
4.1.2 React Standards.....	27
4.1.3 Column Virtualization Performance.....	28
4.2 Estimation:.....	28

(U) Definitions

- **Hook-Driven Architecture** - in React, refers to the systematic use of hooks to manage state, side effects, and reusable logic across components. This approach enhances code readability, keeps components lean, and improves maintainability.
 - **Module Architecture** - Modular architecture, also known as modular design, is a method of subdividing a system into smaller, independent parts called modules or skids, which can be created separately and combined in various configurations. Each module is designed to be scalable, reusable, and interchangeable, enabling architects and builders to modify, expand, or repair structures without affecting the entire system. This approach combines standardization with customization, allowing for both efficiency and creative flexibility.
-

(U) Labels:

- **(C)** - Community Version
 - **(I)** - Planned/Incompleted Feature
 - **(P)** - Premium Version
 - **(Pro)** - Pro Version
 - **(U)** - Unclassified
 - **Highlighted Figures** - Figure outside this document (Figure [0-9].[0-9].[0-9][a-z])
 - EX: **Figure 0.0.0a**
-

1. (U) Investigation Results

1.1 Libraries That Did Not Meet Criteria

1.1.1 HandsonTable

A really good package that is better than FancyGrid. However, there is no row grouping feature, and that is a requirement for CRT's GUI.

1.1.2 Tanstack

Same with Handson table, Tanstack is good, but there are some features missing that are required. (Ex: Tree Data)

1.1.3 RevoGrid

Once again, same issues as above where this package does not have all the required features we need in CRT.

2. (U) MUI X Data Grid

2.1 Introduction:

MUI X is a collection of advanced UI components for complex use cases. Most of MUI X's components are available for free, but more advanced features require a Pro or Premium commercial license. MUI X focuses on making building dynamic, data-intensive apps a breeze. It's mostly designed agnostic. The components can be used with third-party React component libraries that implement different designs.

MUI X Data Grid is built around a modular, hook-driven architecture that delivers high performance, deep customization, and seamless integration with the broader MUI design system. Its core design emphasizes extensibility, virtualization for large datasets, and an open-core model that scales from free community features to advanced enterprise capabilities.

2.2 Main Features:

2.2.1 Cells

This feature lets developers customize the rendered elements and values of a cell.

2.2.1.1 `renderCell()`

The `renderCell()` function is to render any element inside of a cell. This is the only way to render a React component inside a cell, and also the only way to customize a cell's behavior - for example, by adding a click handler. Though powerful, it's also expensive in terms of performance, so it should only be used as a last resort when there are no other means for implementing a specific use case.

2.2.1.2 `valueGetter()`

The `valueGetter()` function is to derive a cell's value from the row data. This is the most performant way to customize the contents of a cell, and it does so without altering the row data itself.

Common use cases include:

- Transforming a value (for example, converting a decimal value to a percentage value)
- Deriving a value from multiple fields (for example, concatenating first name and last name)
- Deriving a value from a nested field (for example, `user.address.city`)

This function is also used internally in the Data Grid to filter, sort, and render (if `renderCell()` or `valueFormatter()` are not provided).

2.2.1.3 `valueFormatter()`

The `valueFormatter()` function is to format a cell's value without changing the underlying row data.

Common use cases include:

- Formatting a date to a custom display format
- Formatting a decimal value to a percentage and appending a % sign
- Formatting a boolean value to Yes or No

Unlike `valueGetter()`, this function only impacts rendering-not internal calculations like filtering or sorting.

2.2.2 Columns

2.2.2.1 Column Visibility (C)

By default, all the columns are visible. The column's visibility can be switched through the user interface in two ways:

- By opening the column menu and clicking the Hide menu item.
- By clicking the Columns menu and toggling the columns to show or hide.

Both are two separate components on the UI that the user can interact with. Both can also be disabled on the GUI to force certain visible columns for users (Ex: User vs Admin columns for a single table).

There are also options to force certain columns to always be visible to the user. Therefore, the user cannot toggle the visibility of that column. This can be seen in **Figure 2.2.2a**, where the checkbox selection for “username” is disabled, but is set to visible.

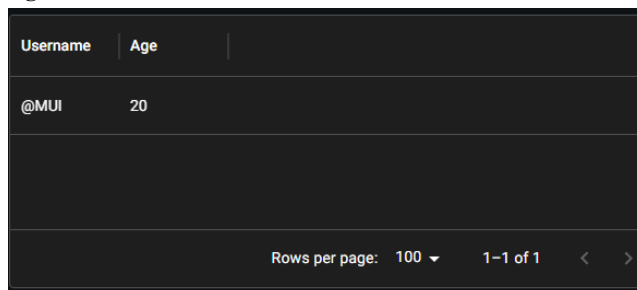
Figure 2.2.2a

2.2.2.2 Column Headers (C)

Headers are handled very similar to Tabulator. You can configure the headers with:

- **headerName**: The title of the column rendered in the column header cell.
- **description**: The description of the column rendered as tooltip if the column header name is not fully displayed.

Figure 2.2.2b

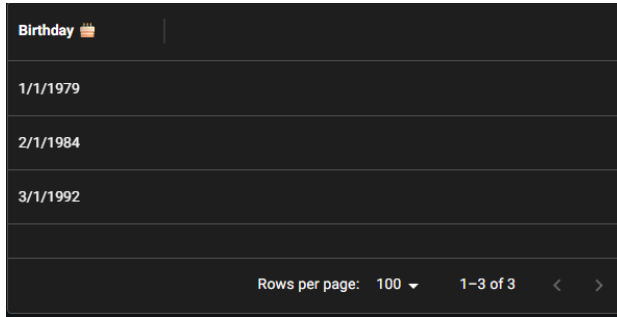


Username	Age
@MUI	20

Rows per page: 100 ▼ 1-1 of 1 < >

Unlike Tabulator, we have the option to add tooltips to the headers to give users more details, and the option to make custom column headers with React components or custom titles like in **Figure 2.2.2c**.

Figure 2.2.2c



Birthday 🎂
1/1/1979
2/1/1984
3/1/1992

Rows per page: 100 ▾ 1-3 of 3 < >

2.2.2.3 Column Menu (C)

Each column header comes with a column menu with quickly accessible Data Grid features like column visibility, sorting, filtering, and others.

It can be accessed by clicking on the 3-dots icon that appears on hover on a header cell or when focusing on a column header and pressing *Ctrl+Enter*

Figure 2.2.2d

The column menu is customizable, where custom components and/or features can be triggered by adding custom options in the menu (**Figure 2.2.2e**). This gives CRT more flexibility with any customizations with column logic, table logic, or cleaner transitions to other features in CRT.

Figure 2.2.2e

2.2.2.4 Column Groups (C)

This feature allows grouping columns in a table into its own groups. This is done in CRT for a limited number of tables, so this feature would be necessary to maintain the look and feel in the current build.

You can define column groups with the *columnGroupingModel* prop. This prop receives an array of column groups.

A column group is defined by at least two properties:

- **groupId:** a string used to identify the group
- **children:** an array containing the children of the group

The children attribute can contain two types of objects:

- leaves with type `{ field: string }`, which add the column with the corresponding field to this group
- other column groups, making it possible to create nested groups

Note: A column can only be associated with one group.

Figure 2.2.2f

Internal	Basic info	Full name	ID	First name	Last name	Age
			☐	Jon	Snow	14
			☐	Cersei	Lannister	31
			☐	Jaime	Lannister	31

Rows per page: 100 1-9 of 9

Each column group can be customizable like any individual column header. However, there is an unique feature with the column groups, where column groups can be collapsable and expandable. This can be seen in **Figure 2.2.2g**.

Figure 2.2.2.g

2.2.2.5 Column Reordering (Pro)

The Data Grid Pro lets users drag and drop columns to reorder them.

Columns are organized according to the order in which they're provided in the columns array. By default, Data Grid Pro lets users reorder columns by dragging and dropping the header cells.

Figure 2.2.2h

2.2.2.6 Column Pinning (Pro)

Column Pinning allows users to keep columns in the Data Grid visible at all times

Pinned columns (also known as sticky, frozen, and locked) are visible at all times while scrolling the Data Grid horizontally.

Users can access this feature through the column menu to pin and unpin columns to either the left or right side. Pinned columns cannot be reordered, except by unpinning and repinning.

Figure 2.2.2i

Certain features, such as checkbox selection and row reordering, add autogenerated columns in the Data Grid. You can pin these by adding `GRID_CHECKBOX_SELECTION_COL_DEF.field` and `GRID_REORDER_COL_DEF.field`, respectively, to the list of pinned columns.

This particular feature is another area that could reduce clicks for the user's convenience in CRT.

Figure 2.2.2j

2.2.3 Filtering

2.2.3.1 Overview

Compared to Tabulator, Data Grid has much more filtering features that gives the user more flexibility and options to filter the data shown to them. The features include what is already implemented in CRT with Tabulator (Header Search Filtering and Column Sorting), and more that are listed below.

The filters can be modified through the Data Grid interface in several ways:

- By opening the column menu and clicking the Filter menu item.
- By clicking the Filters button in the Data Grid toolbar (if enabled).

2.2.3.1.a Excel-Like Filtering (C)

Figure 2.2.3a

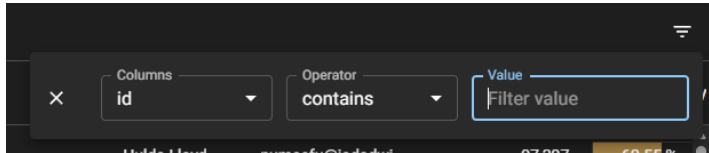
2.2.3.1.b Filter Panel (C)/(I)

⚠ This component is incomplete.
 Currently, the Filter Panel Trigger is the only part of the Filter Panel component available. Future versions of the Filter Panel component will make it possible to compose each of its parts to create a custom filter panel.

In the meantime, it's still possible to deeply customize the panel's subcomponents using custom slots.

The filter panel is enabled by default. Users can trigger the filter panel via the column menu, as well as from the toolbar when `showToolbar` is passed to the `<DataGrid />` component.

Figure 2.2.3b



2.2.3.2 Header Filters (Pro)

By default, users access the Data Grid's filtering features through the filter panel in the toolbar. The header filter feature adds a row to the top of the Data Grid Pro that lets users quickly filter columns directly in place. Filters added through the filter panel are synchronized with header filters, and vice versa.

Figure 2.2.3c

You can override header filter cells individually or across the entire row. This includes removing header filters on certain columns, changing the header filter to be a custom component, etc.

Figure 2.2.3d

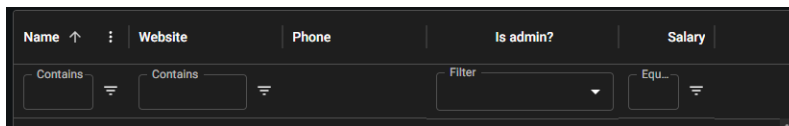


Figure 2.2.3e

2.2.3.3 Multi-Filters (Pro)

This feature lets end users apply multiple filters to the Data Grid simultaneously.

By default, it's only possible to apply one filter at a time to the Data Grid. With the Data Grid Pro, users can apply multiple filters based on different criteria for more fine-grained data analysis. They can apply multiple filters to a single column and multiple columns simultaneously.

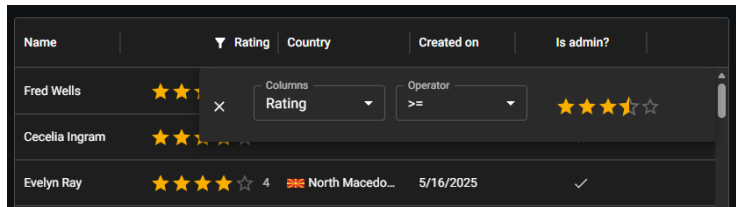
Figure 2.2.3f

2.2.3.4 Custom Filters

2.2.3.4.a Custom Input Components (C)

The value used by the operator to look for needs to be entered by the user. On most column types, a text field is used. However, a custom component can be rendered instead.

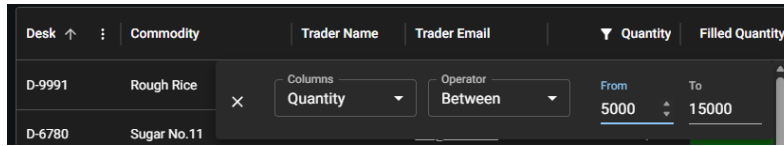
Figure 2.2.3g



2.2.3.4.b Multiple Values Operator (C)

You can create a custom operator which accepts multiple values.

Figure 2.2.3h



2.2.3.5 Sorting (C)

Sorting is enabled by default to the Data Grid users and works out of the box without any explicit configuration. Users can set a sorting rule simply by clicking on a column header. Following clicks change the column's sorting direction. You can see the applied direction on the header's arrow indicator.

Figure 2.2.3i

2.2.3.6 Multi-Sorting (Pro)

The following demo lets you sort the rows according to several criteria at the same time.

By default, users need to hold down the *Ctrl* or *Shift* key while clicking the column header.

Figure 2.2.3j

2.2.3.7 Quick Filter (C)

Quick filter lets users filter rows by multiple columns with a single text input.

By default, the quick filter considers the input as a list of values separated by space and keeps only rows that contain all the values. The quick filter is displayed by default when *showToolbar* is passed to the `<DataGrid/>` component.

Also by default, the quick filter excludes hidden columns.

Figure 2.2.3k

2.2.4 Rows

2.2.4.1 Density (C)

For this feature, DataGrid exposes the density prop which supports the following values:

- **Standard (default)**
- **Compact**
- **Comfortable**

The density prop applies the values determined by the `rowHeight` and `columnHeaderHeight` props, if supplied. The user can override this setting with the optional toolbar density selector.

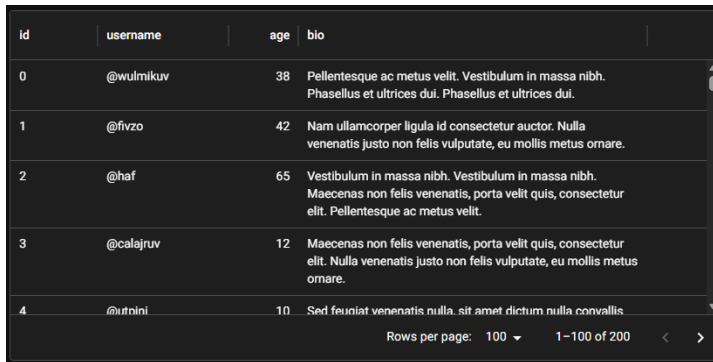
You can create a custom toolbar with a density selector that lets users change the density of the DataGrid, as shown in **Figure 2.2.4a**.

Figure 2.2.4a

2.2.4.2 Dynamic Row Heights (C)

Instead of a fixed row height, you can let the Data Grid calculate the height of each row based on its content.

Figure 2.2.4b



id	username	age	bio
0	@wulmiku	38	Pellentesque ac metus velit. Vestibulum in massa nibh. Phasellus et ultrices dui. Phasellus et ultrices dui.
1	@fivzo	42	Nam ullamcorper ligula id consectetur auctor. Nulla venenatis justo non felis vulputate, eu mollis metus ornare.
2	@haf	65	Vestibulum in massa nibh. Vestibulum in massa nibh. Maecenas non felis venenatis, porta velit quis, consectetur elit. Pellentesque ac metus velit.
3	@calajruv	12	Maecenas non felis venenatis, porta velit quis, consectetur elit. Nulla venenatis justo non felis vulputate, eu mollis metus ornare.
4	@utrolni	10	Sed feuniat venenatis nulla. sit amet dictum nulla convallis

Rows per page: 100 1-100 of 200

The dynamic row height implementation is based on a lazy approach, which means that the rows are measured as they are rendered. Because of this, you may see the size of the scrollbar thumb changing during scroll. This side effect happens because a row height estimation is used while a row is not rendered, then this value is replaced once the true measurement is obtained.

You can configure the estimated value used by passing a function to the `getEstimatedRowHeight` prop. If not provided, the default row height of `52px` is used as an estimation. It's recommended to pass this prop if the content deviates too much from the default value.

Figure 2.2.4c

2.2.4.3 High Frequency (Pro)

Whenever the rows are updated, the Data Grid has to apply sorting and filters. This can be a problem if you have high-frequency updates. To maintain good performance, the Data Grid lets you batch the updates and only apply them after a period of time. You can use the `throttleRowsMs` prop to define the frequency (in milliseconds) at which rows updates are applied.

When receiving updates more frequently than this threshold, the Data Grid will wait before updating the rows.

Figure 2.2.4d

2.2.4.4 Master-Detail Row Panels (Pro)

"Master-detail" is a design pattern for organizing information in which the "master" area lists the data and the "detail" sections provide further information about each item.

The Data Grid Pro provides the tools to implement master-detail row panels. These are useful whenever you need to give end users additional information about row items without navigating away from the Grid. A common example of this pattern is found in many email clients—users can click on an email from the master list to see its contents (details) as well as actions they can take such as replying, forwarding, and deleting.

Figure 2.2.4e

As an alternative to the built-in row editing feature, you can render a form component inside the detail panel so users can edit the row values.

Figure 2.2.4f

2.2.4.5 Row Pinning (Pro)

Row pinning allows users to keep rows in the Data Grid visible at all times. Pinned rows (also known as sticky, frozen, and locked) are visible at all times while scrolling the Data Grid vertically. Pinned rows are not affected by sorting, filtering, or pagination—they remain pinned regardless of how these features are applied. You can pin rows to the top or bottom of the grid.

Pinned rows do not support the following features:

- selection
- row grouping
- tree data
- row reordering
- master-detail row panels

You can control which rows are pinned by making dynamic changes to `pinnedRows`. **Figure 2.2.4g** uses the actions column type to provide buttons that let the user pin a row to the top or bottom of the Grid.

Figure 2.2.4g

2.2.4.6 Row Reordering (Pro)

The Data Grid Pro lets users drag and drop rows to reorder them.

By default, users cannot manually rearrange rows in the Data Grid. With the Data Grid Pro, you can give them the ability to drag and drop rows to reorder them.

You can capture reordering events, where the developer can access the ordering data in the table. This can be used very effectively for the OpReqs tool in CRT to reduce clicks.

Figure 2.2.4h

2.2.4.7 Row Selections

2.2.4.7.a Cell Selection (P)

By default, the Data Grid lets users select individual rows. With the Data Grid Premium, you can apply the `cellSelection` prop to let users select individual cells or ranges of cells.

Figure 2.2.4i

Desk	Commodity	Trader Name	Trader Email	Quantity
D-6158	Coffee C	Lydia Butler	latpuyba@juken.ga	87,619
D-5472	Milk	Oscar Beck	hujuweej@lo.ic	70,758
D-9946	Soybean Meal	Inez Turner	luoh@up.us	65,260
D-7862	Oats	Sylvia Vega	raswuc@datsl.ny	31,994
D-6957	Frozen Concentrated Or...	Henrietta Mart...	cuflu@ina.uy	69,795
D-4998	Sugar No. 11	Jane Obrien	feiroz@naftem.me	58,933

Total Rows: 10

2.2.4.7.b Single Selection (C)

Single row selection comes enabled by default for the MIT Data Grid component

Figure 2.2.4j

Desk	Commodity	Trader Name	Trader Email	Quantity
D-539	Frozen Concentrated Or...	Garrett Ray	tokanta@ziv.ci	64,770
D-2032	Oats	Lillian Reid	durde@ld.gf	24,935
D-9943	Soybean Oil	Lloyd Valdez	kud@fokidjut.io	51,990
D-4373	Sugar No. 14	Manuel Bass	novusgom@qa.ar	53,848
D-4360	Soybeans	Cory Palmer	zule@fupsu.bs	93,994
D-497	Soybean Meal	Thomas Valdez	anu@uzecup.bz	20,525

1 row selected Rows per page: 100 1-10 of 10

2.2.4.7.b Multi Selection (Pro)

Figure 2.2.4k

Desk	Commodity	Trader Name	Trader Email	Quantity
D-1206	Corn	Lillian Snyder	lu@atinu.mo	90,863
D-7748	Soybean Meal	Jeffery Robbins	hama@un.gd	70,512
D-2896	Adzuki bean	Christina Bass	zenpuwo@besujl.ga	72,266
D-300	Milk	Verna Graves	eltodo@rituvaj.om	85,868
D-953	Wheat	Nathaniel Tho...	babuj@igemezu.bb	75,235
D-7617	Soybean Meal	Harriett Clavton	noh@ilikuwa.re	53,425

3 rows selected 1-10 of 100

2.2.4.7.c Disable selection on certain rows (C)

Figure 2.2.4l

	Desk	Commodity	Trader Name	Trader Email	Quantity
☐	D-7849	Milk	Bettie Warren	cila@lumali.bd	49,899
☒	D-1290	Milk	Christine Und...	pow@fewudta.mn	84,303
☐	D-6625	Soybean Oil	Ethan Dunn	wohiv@larvawe.vi	28,185
☐	D-5488	Soybeans	Helena Norman	suwurt@nov.ag	38,834
☐	D-5646	Adzuki bean	Bernard Knight	uzuso@tarta.no	61,075
1 row selected Rows per page: 100 1-100 of 100					

2.2.4.7.d Row selection with filtering (C)

By default, when the rows are filtered the selection is cleared from the rows that don't meet the filter criteria. To keep those rows selected even when they're not visible, set the *keepNonExistentRowsSelected* prop.

2.2.4.8 Row Spanning (C)

By default, each cell in a Data Grid takes up the height of one row. The row spanning feature makes it possible for a cell to fill multiple rows in a single column. To enable this, passing the *rowSpanning* prop to the Data Grid will automatically merge consecutive cells with repeating values in the same column.

Row spanning can be used in conjunction with column spanning to create cells that span multiple rows and columns simultaneously

Figure 2.2.4m

Name	Designation	Department	Age	
Andrew Clark	React Engineer	Engineering	25 yo	
	Technical Interviewer	Human resource		
Cynthia Duke	Technical Team Lead	Engineering	25 yo	
Jordyn Black	React Engineer		31 yo	
Rene Glass	Ops Lead	Operations	31 yo	

2.2.4.9 Column Virtualization (C)

By default, the virtualization of the columns is disabled to force all columns to be rendered at the same time and calculate the row height correctly. However, this can lead to poor performance when rendering a lot of columns.

If you need column virtualization, you can set the *virtualizeColumnsWithAutoRowHeight* prop to true. With this approach, the Data Grid measures the row height based on the visible columns. However, the row height might change during horizontal scrolling.

2.3 Advanced Features:

2.3.1 Aggregation (P)

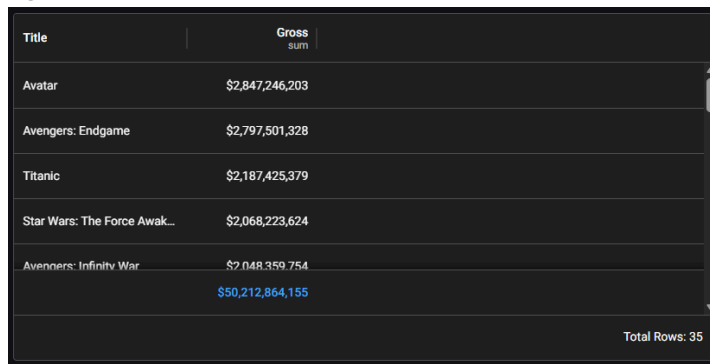
Aggregation allows users to combine row values at the bottom of the table.

The Data Grid Premium provides tools to give end users the ability to aggregate and compare row values. It includes built-in functions to cover common use cases such as sum, average, minimum, and maximum, as well as the means to create custom functions for all other needs.

Users can aggregate rows through the Data Grid interface by opening the column menu and selecting from the items under Aggregation. The aggregated values are rendered in a footer row at the bottom of the Grid.

When row grouping is enabled, aggregated values can be displayed in the grouping rows as well as the top-level footer.

Figure 2.3.1a



Title	Gross sum
Avatar	\$2,847,246,203
Avengers: Endgame	\$2,797,501,328
Titanic	\$2,187,425,379
Star Wars: The Force Awak...	\$2,068,223,624
Avengers: Infinity War	\$2,048,359,754
	\$50,212,864,155

Total Rows: 35

2.3.2 AI Assistant (P)

2.3.2.1 Overview

The AI Assistant feature lets users interact with the Data Grid component using natural language. Type a prompt like "sort by name", "show amounts larger than 1000", or even make more complex queries like "which customers brought the most revenue the past year" in the prompt input field and the Data Grid will update accordingly. In supported browsers, users can also prompt the assistant using their voice.

To increase the accuracy of the language processing, provide example values for the available columns via one of the methods below:

- **Provide custom examples** - Use the *examples* property on items of the columns array to provide custom examples as context for prompt processing. The *examples* property should contain an array of possible values for each respective column.
- **Use row data for examples** - Pass the *allowAiAssistantDataSampling* prop to use row data to generate examples. This is useful if you're dealing with non-sensitive data and want to skip creating custom examples for each column. Data is collected randomly at the cell level, which means that the examples for a given column might not come from the same rows.
- **Using server-side data**
- **Data visualization** - AI Assistant analyzes the query to determine if it is helpful to visualize the results

Natural language prompts must be processed by a service to understand what kinds of state changes must be applied to the Data Grid to match the user's request. You can use MUI's processing service or build your own.

The Data Grid exposes elements of the AI Assistant feature so you can build your own prompt processing service:

- The *aiAssistant* API for processing the prompt results and updating state
- The *unstable_gridDefaultPromptResolver()* method for passing the prompt and context with the necessary headers to the processing service

You can integrate these elements with your own custom components and methods to suit your specific use case.

2.3.2.2 Prompt Field

Provide users with a prompt field to interact with the AI assistant. You can use the Prompt Field component directly if you want to build your own UI for the AI Assistant.

Figure 2.3.2a

2.3.2.3 AI Assistant Panel (I)

⚠ This component is incomplete.

Currently, the AI Assistant Panel Trigger is the only part of the AI Assistant Panel component available. Future versions of the AI Assistant Panel component will make it possible to compose each of its parts for full customization.

You can use the AI Assistant Panel Trigger and Toolbar components when you need to customize the AI assistant panel trigger, or when implementing a custom toolbar.

Figure 2.3.2b

2.3.3 List View (Pro)

This feature displays data in a single-column list view for a more compact Data Grid on smaller screens and mobile devices.

A typical data grid built for desktop devices can be difficult to use on a mobile device. The Data Grid Pro provides a single-column list view format to give users a better experience on smaller screens.

This can be implemented for tables with 2-3 columns of records, or custom components that could transition into this type of component. However, this feature is most likely not necessary and is a very niche feature for CRT to implement.

Figure 2.3.3a

Icon	Name	Role
	Roy Leonard	Health Therapist
	Eunice McDonald	Recreational Director
	Hallie Peterson	Recreational Director
	Ian Woods	Joint Venture Manager
	Mabel Beck	Communications Engineer
	Helena Leonard	Physician Assistant
	Leah Goodwin	Nursing Administrator
	Cornelia Rowe	Multimedia Director

Total Rows: 20

2.3.4 Pivoting (P)

The Pivoting feature purpose is to rearrange rows and columns to view data from multiple perspectives.

Pivoting gets its name from the idea of rotating data—for example, you might take a flat list of transactions with columns for product, region, and sales, and then pivot it to compare these values in different ways. With pivoting, you could turn the unique product names into rows and the regions into columns, and aggregate the sales values to see the total sales per product per region.

2.3.4.1 Rows

The Rows menu defines how the data will be grouped vertically after pivoting. Each unique value in the fields selected for rows will become a new row in the pivot Data Grid, and if multiple fields are selected for rows, the rows will be grouped in the order of the selected fields.

Figure 2.3.4a

Product	
Apples (4)	
Oranges (4)	

Total Rows: 2

Pivot

Search fields

Region +

Quarter +

Sales +

Profit +

Rows 2

☒ Product +

☐ Size +

Columns

Drag here to create columns

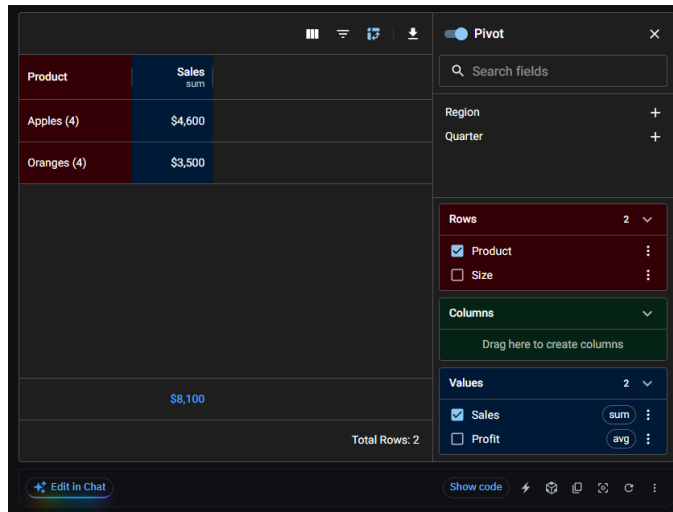
Values

Drag here to create values

2.3.4.2 Values

The Values menu defines what data will be displayed in the cells of the pivot Data Grid. These are typically numeric fields that can be aggregated (often into a sum, an average, or a total count), and if multiple fields are selected for values, each field will add a new aggregated column to the pivot Data Grid.

Figure 2.3.4b



2.3.4.3 Columns

The Columns menu defines how the data will be grouped horizontally after pivoting. Each unique value in the fields you select for columns will create a new column group in the pivot Data Grid. If multiple fields are selected for columns, the columns will be grouped in the order of the selected fields.

Figure 2.3.4c

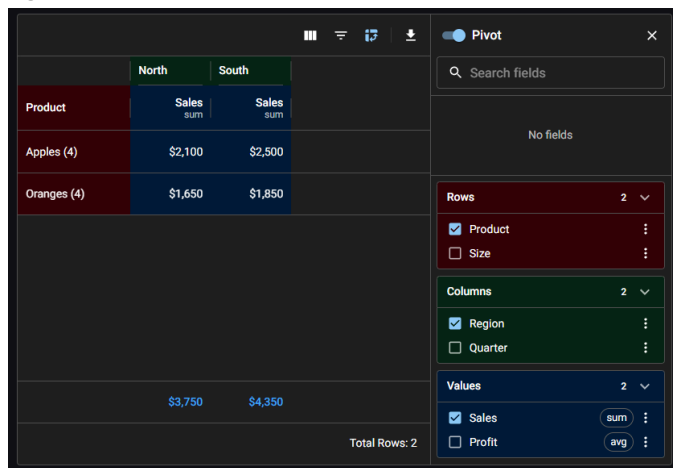


Figure 2.3.4d

The screenshot shows the Data Grid Premium interface. On the left is a pivoted table with the following data:

		North		
		Q1		Q2
Group		Sales sum	Profit avg	Sales sum
Apples (4)		\$1,000	100%	\$1,100
L (3)		\$1,000	100%	\$1,100
M (1)				
Oranges (4)		\$800	80%	\$850
M (2)		\$800	80%	\$850
		\$1,800	90%	\$1,950

At the bottom right of the table, it says "Total Rows: 2". To the right of the table is the "Pivot" panel, which is open. It has a "Search fields" input at the top. Below it, there are three sections: "Rows" (2 items), "Columns" (2 items), and "Values" (2 items). The "Rows" section contains "Product" and "Size". The "Columns" section contains "Region" and "Quarter". The "Values" section contains "Sales" (sum) and "Profit" (avg).

The Data Grid Premium's pivoting feature lets users transform the data in their grid by reorganizing rows and columns, creating dynamic cross-tabulations of data. This makes it possible to analyze data from different angles and gain insights that would be difficult to see in the default grid view.

Pivoting is enabled by default and can be accessed through the icon in the toolbar. In **Figure 2.3.4a**, The pivot panel is already open and some pivoting parameters have been set. Using the Pivot switch at the top of the panel, the user can toggle pivoting off and on. You can drag and drop existing columns in the Rows, Columns, and Values dropdown menus to change how the data is pivoted.

Figure 2.3.4e

There are certain sections of CRT that can benefit the user to use pivoting, especially for giant tables with the potential to display row and column groupings. There are ways to initialize the pivoting information, so CRT can implement something similar to the *Quick Filters* to help users preset and digest large data sets quickly.

2.3.5 Row Grouping (P)

This feature groups rows together based on column values in the Data Grid.

The Data Grid Premium provides a row grouping feature to create subsets of rows based on repeated column values or custom functions.

In CRT, this feature is the same as Tabulator's *group-by* feature. We use this to group all sorts of records based on a certain column's entries.

Figure 2.3.5a

2.3.6 Tree Data (Pro)

Similar to row grouping, tree data is used to render rows with parent-child relationships in the Data Grid.

Trees are hierarchical data structures that organize data into parent-child relationships. The Data Grid Pro can use tree data to render rows that conform to this pattern.

In CRT, this feature is the same as Tabulator's *tree data* feature. We use this to show the latest version of a record as the parent, and collapse/expand the previous versions as the children rows.

Figure 2.3.6a

2.4 Components:

2.4.1 Column Panel (I)

⚠ This component is incomplete.

Currently, the Columns Panel Trigger is the only part of the Columns Panel component available. Future versions of the Columns Panel component will make it possible to compose each of its parts to create a custom columns panel.

In the meantime, it's still possible to deeply customize the panel's subcomponents using custom slots.

The columns panel is part of the column visibility feature and is enabled by default. Users can trigger the columns panel via the column menu, as well as from the toolbar when `showToolbar` is passed to the `<DataGrid />` component.

You can use the Columns Panel Trigger and Toolbar components when you need to customize the columns panel trigger, or when implementing a custom toolbar.

Figure 2.4.1a

2.4.2 Export

Export the rows in CSV or Excel formats, or use the browser's print dialog to print or save as PDF.

Following are the available export options:

- **Print (C)**
- **CSV (C)**
- **Clipboard (C)**
- **Excel (P)**

2.4.2.1 CSV (C)

By default, CSV is used to export tables in Data Grid. There are no options to modify or insert records before converting the table into a CSV and before triggering the download blob from the GUI. So, CRT would need to use our custom export component in the toolbar, or use the *Excel* feature under Data Grid Premium for classification purposes.

Figure 2.4.2a

Figure 2.4.2c

The screenshot displays a web application interface. At the top, there's a header bar with a logo and several utility icons (list, filter, refresh, download, search). Below the header, a table is shown with 10 rows and 3 columns. The first column has a dropdown menu open, revealing options like 'Column groups', 'Column spanning', 'Column resizing', 'Column reorder', 'Column pinning', and 'Row (5)'. The second column contains labels such as 'Community' or 'Pro'. The third column features status indicators, either an 'X' or a checkmark. To the right of the table, a vertical sidebar lists various navigation items: Home, Users, Products, Orders, Customers, Suppliers, Categories, Roles, Permissions, Settings, and Audit Log.

Feature	plan	develop
▼ Column (5)		
Column groups	Community	X
Column spanning	Community	X
Column resizing	Pro	✓
Column reorder	Pro	✓
Column pinning	Pro	✓
Row (5)		
Total Rows: 10		

2.4.2.1 Excel (P)

This feature relies on **exceljs (npm package)**. With Excel export, users can translate the column types and tree structures of a Data Grid to an Excel file.

Columns with types 'boolean', 'number', 'singleSelect', 'date', and 'dateTime' are exported in their corresponding type in Excel. Please ensure the rows values have the correct type, you can always convert them as needed. The excel export option will appear in the default toolbar export menu by passing the `showToolbar` prop to `<DataGridPremium />`.

You can customize the document using two callback functions:

- *exceljsPreProcess* called before adding the rows' dataset.
- *exceljsPostProcess* called after the dataset has been exported to the document.

Both functions receive `{ workbook, worksheet }` as input. These are exceljs objects that let you directly manipulate the Excel file. Thanks to these two methods, you can modify the metadata of the exported spreadsheet. You can also use it to add custom content on top or bottom of the worksheet

Figure 2.4.2d

	A	B	C
1	This is an helping document for the MUI-X team.		
2	Please refer to the doc for up to date data		
3			
4	Feature	plan	developed
5	Pagination	Community ▾	

There is also a built-in feature to use web workers for Excel downloads. Instead of generating the Excel file in the main thread, you can delegate the task to a web worker. This method reduces the amount of time that the main thread remains frozen so users can interact with the grid while the data is exported in background.

Since the main thread is not locked while the data is exported, it is important to give feedback for users that something is in progress. You can pass a callback to the *onExcelExportStateChange* prop and display a message or loader for the user.

Figure 2.4.2e

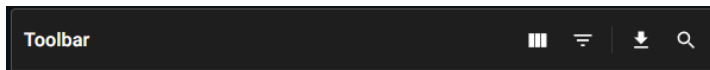
2.4.3 Toolbar

This feature adds custom actions and controls to the Data Grid. The default toolbar can be enabled by passing the *showToolbar* prop to the `<DataGrid />` component.

You can use the Toolbar and various other Data Grid components when you need to customize the toolbar:

- Columns Panel
- Filters Panel
- Export
- Settings
- Quick Filter
- Etc...

Figure 2.4.3a



2.5 Customization:

2.5.1 Overlays

Data Grid has different overlay features for a table.

The Data Grid supports 3 loading overlay variants out of the box:

- **Skeleton:** an animated placeholder of the Data Grid.
- **Linear-progress:** an indeterminate linear progress bar.
- **Circular-progress:** a circular loading spinner.

2.5.1.1 Skeleton

An animated placeholder of the Data Grid.

Figure 2.5.1a

2.5.1.2 Linear Progress

An indeterminate linear progress bar.

Figure 2.5.1b

2.5.1.3 Circular Progress

A circular loading spinner.

Figure 2.5.1c

2.5.1.4 Custom Component

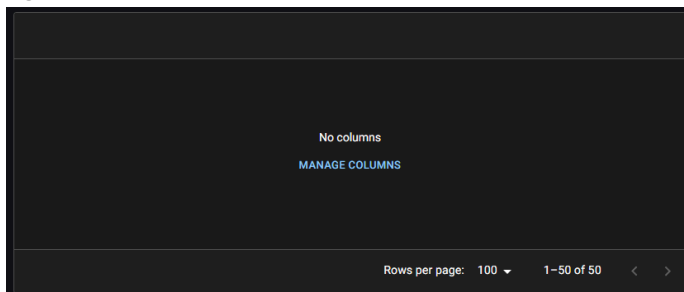
If you want to customize the no-rows overlay, a component can be passed to the *loadingOverlay* slot.

Figure 2.5.1d

2.5.1.5 No Columns Overlay

The no-columns overlay is displayed when the Data Grid has no columns, or when all columns are hidden. The "Manage columns" button is displayed when all columns are hidden and `disableColumnSelector` is false.

Figure 2.5.1e



2.5.1.6 No Rows Overlay

The no-rows overlay is displayed when the Data Grid has no rows.

Figure 2.5.1f

Desk	Commodity	Trader Name	Trader Email	Quantity
No rows				
Rows per page: 100 ▾ 0-0 of 0 < >				

2.5.1.7 No Results Overlay

The no-results overlay is displayed when the Data Grid has no results after filtering.

Figure 2.5.1g

abc				
Desk	Commodity	Trader Name	Trader Email	Quantity
No results found.				
Rows per page: 100 ▾ 0-0 of 0 < >				

2.6 Concerns:

- Can be very complex for developers to integrate with
- A lot of features. Therefore, some features do not work with others in Data Grid.
- Might have security issues with this package (Not entirely sure, but if AG Grid was approved, then I think Data Grid should be okay)

2.7 Estimation:

- 5-6 months

Reasons:

A lot of features are plug and play, and have a very similar setup to AG Grid. We could reuse the AG Grid demo code to implement Data Grid.

Data Grid is meant for high-performance virtualization:

- Designed to handle tens of thousands to millions of cells efficiently.
- Uses a virtual scroller to render only what's visible, minimizing DOM load.
- Pro and Premium tiers enhance performance further for enterprise-scale datasets.

Deep Integration with the MUI design system:

- Fully themeable using Material UI's theming engine.
- Works seamlessly with other MUI X components (charts, date pickers, etc.).
- Can also be customized to match any design system, not just Material UI.

Intuitive, TypeScript-First API:

- Fully written in TypeScript, providing strong typing and predictable behavior.
- API is designed to be declarative and React-idiomatic (props-driven, controlled/uncontrolled modes).

3. (U) FancyGrid

3.1 Introduction

FancyGrid is designed as a modular, enterprise-grade JavaScript data grid that integrates seamlessly with charting libraries, supports multiple frameworks (JS, Angular, React, Vue), and provides a unified system for data handling, UI components, and server communication. Its architecture centers on auto-loading modules, flexible data sources, and rich built-in features like sorting, filtering, CRUD, theming, and chart integration.

This package is very similar to Tabulator, but has more features and flexibility with React components.

3.2 Features

Since FancyGrid is very much like Tabulator, has features that are very similar to Data Grid, and to reduce repeating informations, here is just a list of features that are included:

- **Cells**
 - valueGetter()
 - valueFormatter()
 - Internal or custom (like Tabulator's logic)
- **Columns**
 - Custom Headers
 - Grouped Headers
 - Column Menus
- **Filtering**
 - Custom Filters
 - Header Filters
 - (Excel-like Filtering available)
 - Sorting
 - Multi Sorting
 - Row Expander (Master-Detail Row Panels)
 - Row Dragging
- **Rows**
 - Context Menu
 - Single Selection
 - Multi Selection
 - Row Grouping
 - Tree Data
- **Components**
 - Export (CSV and Excel)

3.3 Demo Links

Unlike Data Grid, the documentation for FancyGrid does not include mini demos/builds of individual features, However, here are some demo links that showcases the majority of the features FancyGrid provides.

Links:

- <https://www.fancygrid.com/fg-grid/gallery/grouping/>
 - <https://www.fancygrid.com/fg-grid/gallery/big-data/>
 - <https://www.fancygrid.com/fg-grid/gallery/theme-dark/>
 - <https://www.fancygrid.com/dashboards/staff/>
 - <https://www.fancygrid.com/dashboards/filter-form/>
-

3.4 Concerns

- A lot less customization options.
 - Way more maintenance of tables in the future.
 - Documentation is barebones.
 - FancyGrid seems like it would end up running into similar development issues as Tabulator. It looks to have the same hindrance and coding gymnastics for development.
 - Pure Javascript package.
 - No idea how well it does with React Testing Library and BrowserLibrary (Needs IDs appended to row element)
-

3.5 Estimation

- 6-8 months

Reasons:

FancyGrid is a barebone package that gives us what tabulator has, but a bit more. If CRT only needs certain additional features like header filter customization for excel-like filtering, then FancyGrid would be an okay option. Either way, implementing FancyGrid will not be simple, and would require some planning ahead of time.

4. (U) Internal CRT Table Component

4.1 Concerns:

- Lack of a GUI SME
- Not enough dedicated GUI developers on the project to solely work on a table component
- A lot of design and testing needed
- Development time will be way longer than integrating a new package. (Development of table + Integration for each service's GUI)

4.1.2 React Standards

React has well-established conventions around component structure, state flow, hooks, and rendering patterns, and when those aren't understood, developers often reinvent solutions, mix paradigms, or introduce subtle bugs that are hard to trace. It's not just about knowing React syntax; it's about understanding the expectations of the ecosystem—predictable component boundaries, clean separation of concerns, and consistent use of hooks and effects. Without that shared foundation, collaboration slows down and the overall architecture becomes harder to maintain, even when the team is working hard.

4.1.3 Column Virtualization Performance

By default, React would force all columns to be rendered at the same time and calculate the row height correctly. However, this can lead to poor performance when rendering a lot of columns.

To circumvent that issue, we would need column virtualization logic that would measure the row height based on the visible columns. However, the row height might change during horizontal scrolling.

MUI X Data Grid and AG Grid have implemented a *Column Virtualization* feature for this exact problem.

4.2 Estimation:

- 8-12 months
 - Including all necessary features, testing, integration in all services, debugging, etc.